

Human Activity Recognition based on Mobile/Wearable Devices

Ambient Intelligence and Domotics

Master Degree: Artificial Intelligence for Science and Technology

Gabriele Civitarese

EveryWare Lab, Department of Computer Science

Università degli Studi di Milano

Copyright

These slides are from the teachers of this course. All the material is subject to copyright and cannot be redistributed without consent of the copyright holder. The same holds for audio and video-recordings of the classes of this course.

HAR on mobile/wearable devices

- Mobile and wearable devices are ubiquitous in our lives
- They are equipped with several sensors that can be used to keep track about our activities
- Several applications, including:
 - healthcare
 - well-being
 - sport



HAR on mobile/wearable devices: Existing Commercial Solutions

Commercial Solutions



Services offered by commercial solutions

- Lifestyle monitoring
- Sleep quality



- Activity Recognition based on visited places

Activity Recognition API

- Both Android and iOS already include API to recognize users' activity (since several years!)
- It is possible to use these APIs to create context-aware applications

Activities recognized by APIs on mobile OS



- IN_VEHICLE
- ON_BICYCLE
- ON_FOOT
- WALKING
- RUNNING
- TILTING
- STILL
- UNKNOWN



- stationary
- walking
- running
- automotive
- cycling
- unknown

Limits of existing solutions

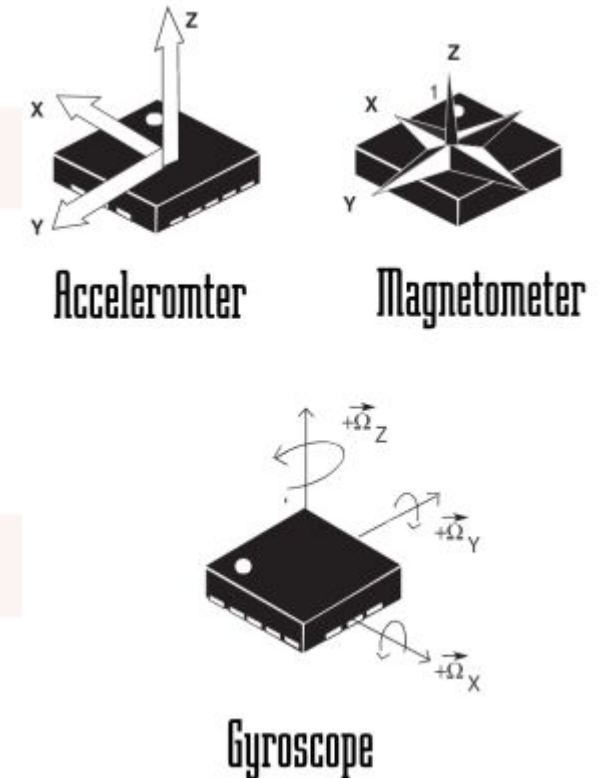
- Limited set of activities
- Recognition rate often not accurate
- The HAR model is not personalized on the end user!

HAR is still a hot research topic due to its many challenges

Sensors of Mobile/Wearable Devices

Inertial sensors

- **Accelerometer:** it provides acceleration values on three axes.
- **Gyroscope:** it provides angular speed on three axis. It reveals changes in direction.
- **Magnetometer:** it provides the information on the magnetic field on three axes. Changes of magnetic field may provide hints about the activity



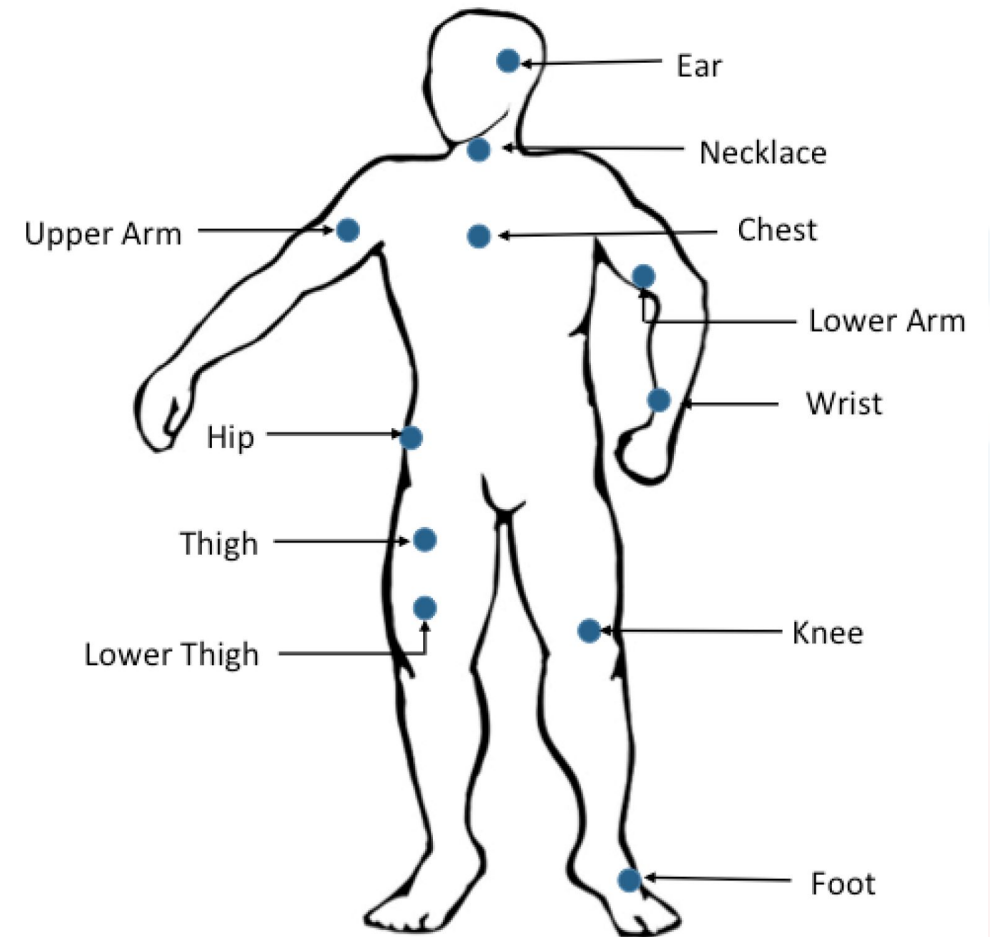
Inertial sensors are the most considered for HAR on mobile/wearable devices, since they reveal physical body movements

Other sensors

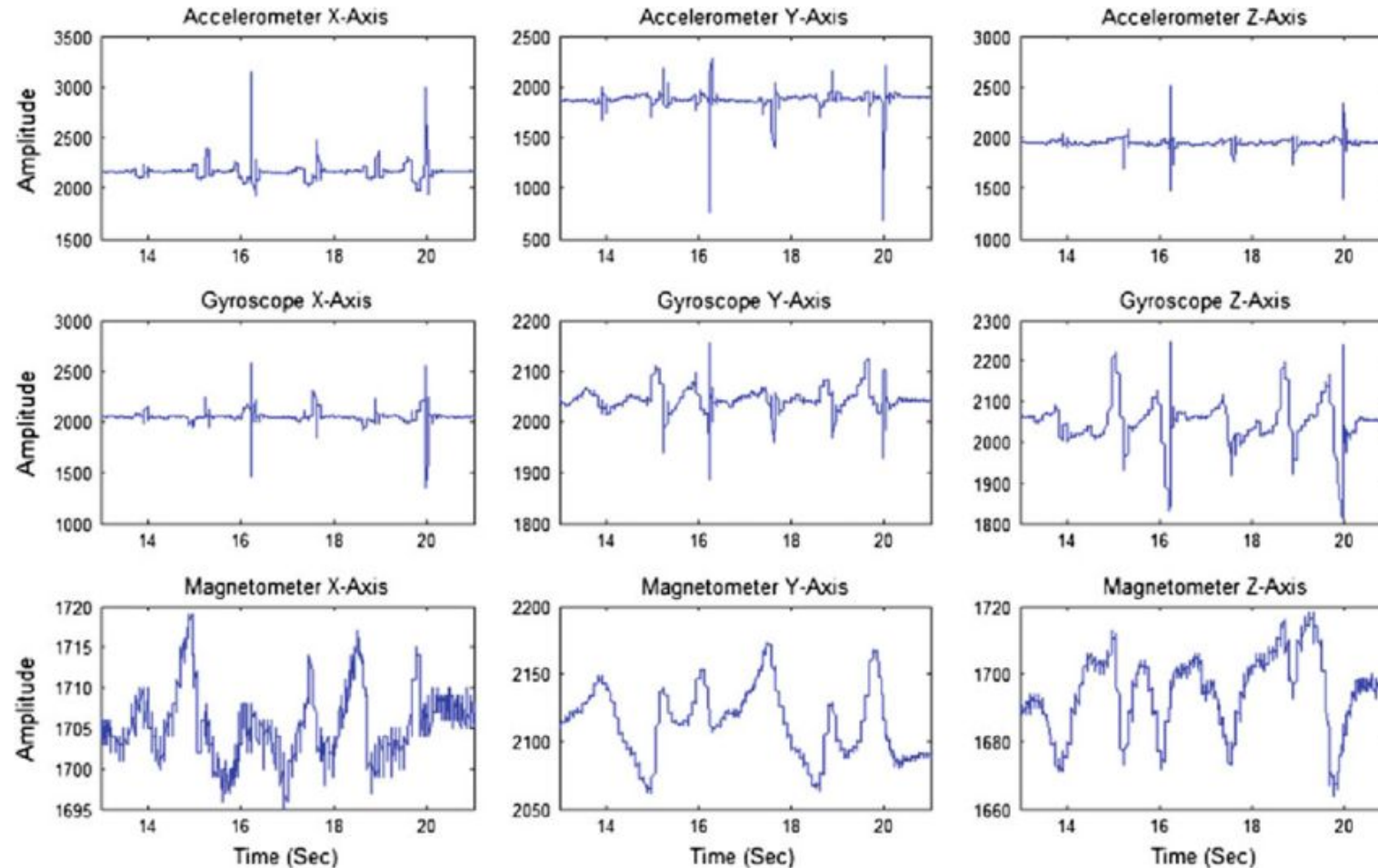
- Other sensors of mobile/wearable devices may provide useful contextual information to better recognize activities, including
 - localization sensors (e.g., GPS)
 - light sensor
 - microphone
 - proximity sensor
 - physiological sensors (e.g., heart rate sensors)

Devices placement

- Wearable/mobile devices may be placed in many different positions, based on the technology!
- The most common setting considered in HAR is
 - smartphone in the pocket
 - smartwatch on the wrist



Inertial sensors raw data



Raw data: how they really look like

●●●○ AT&T 3:13 PM

[Back](#) Lin.Acc: 145 samples

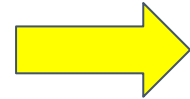
#	Time	X value	Y value	Z value
1	0.021	0.2430	0.1908	-0.6087
2	0.051	0.1955	0.0411	0.0578
3	0.086	0.0485	0.0693	0.1990
4	0.121	0.1086	0.2563	0.0024
5	0.151	0.0126	0.0690	0.0639
6	0.186	-0.1148	0.1831	-0.0423
7	0.221	-0.0488	-0.0535	-0.1411
8	0.251	-0.1029	-0.0412	-0.0522
9	0.286	-0.0660	-0.0438	-0.1654
10	0.321	-0.0078	0.1225	-0.0912
11	0.351	0.0958	0.1212	-0.1178
12	0.386	-0.0370	0.0784	-0.0523
13	0.416	-0.2539	-0.0917	-0.2029
14	0.451	-0.6581	-0.4578	-0.5947
15	0.486	-0.8516	-0.5882	-0.7221
16	0.521	-0.7580	-0.1352	-0.9090
17	0.551	-0.1778	0.0620	0.1805
18	0.586	0.3669	0.4592	0.7906
19	0.616	0.9646	0.8114	0.3629

HAR pre-processing

Pre-processing pipeline



Inertial sensors from
mobile/wearable
devices



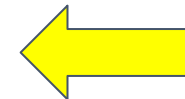
Data cleaning



Segmentation



Feature Extraction



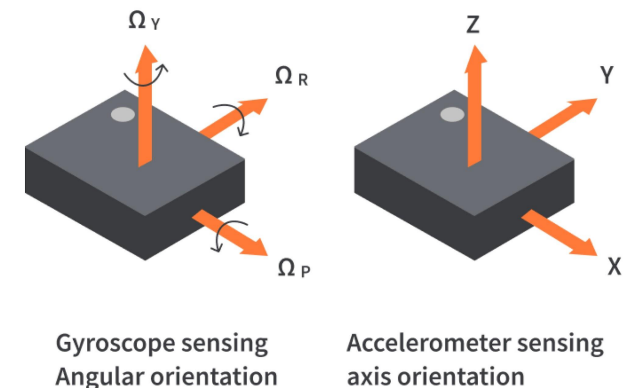
**Feature
Vector**

Data cleaning

- Inertial sensors data are noisy, filters are often required to maintain only informative part of the signal (*smoothing!*)
 - e.g., median filter, low-pass filter, gaussian filter
- Sometimes it is also useful to exclude the gravitational component from acceleration data
 - the result is called *linear acceleration* and it better represents actual users' movements

Linear Acceleration

- The acceleration obtained from the accelerometer also incorporates the **gravity** component
- Filtering out the constant gravity force makes it possible to better discriminate the specific movements/gestures performed by a subject
- This requires combining acceleration and orientation (derived by gyroscope)
- Idea:
 - based on the orientation of the device, subtract to each axis the gravitational component



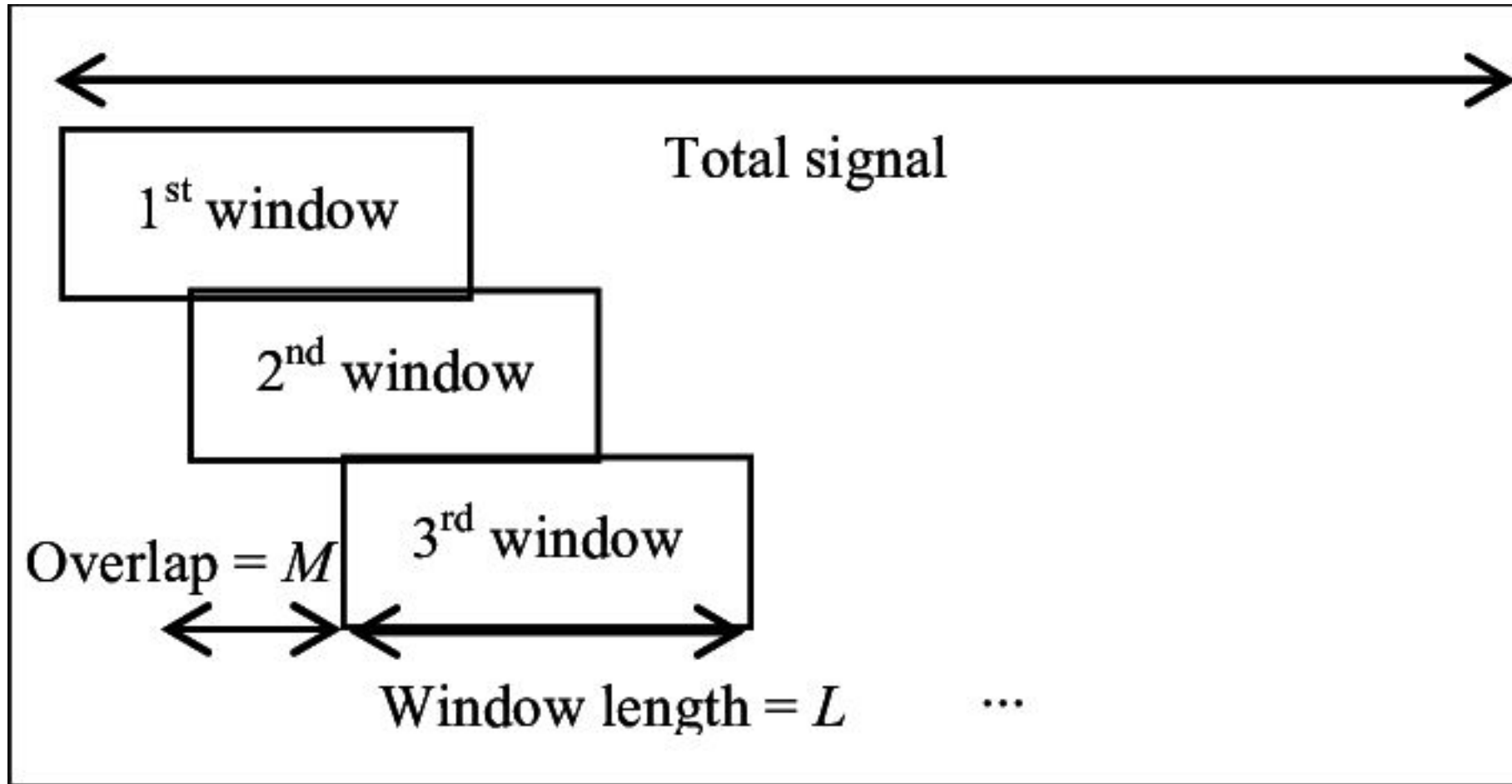
Segmentation on HAR

- Segmentation is performed considering the multivariate time series:
 - each device may generate data from different sensors (e.g., accelerometer, gyroscope)
 - each sensor usually provides multi-channel data (e.g., 3 axis of accelerometer)
 - each user may wear multiple mobile devices

Note that each sensor may have its sampling rate.

Sensor data should be temporally aligned and the clocks of different devices should be synchronized.

Sliding window segmentation



Sliding window parameters

- Two main parameters need to be tuned
 - segment length
 - overlap
- The choice of these parameters strictly depends on the target activities
- Overlap is crucial to capture transitions between activities

Window size

- Simple physical activities (e.g., running, walking, standing, etcetera) requires short time windows
 - e.g. *3 seconds with 50% Overlap*
- More complex activities may require longer window sizes
- However, the choice of the window may also be chosen based whether there are “real-time” requirements
 - trade-off between system’s responses delay and accuracy

About window sizes

Category (Activities)	W_{min_size} (s)	W_{max_perf} (s)
Arms (19–25)	0.5–1	0.75–2.25
Legs (26–30)	0.25–1.75	1.25–5.75
Trunk (12,15,17)	0.5–1.25	0.75–6.25
Trunk + arms (9–11,13,16)	0.5–1.25	1–4.5
Trunk + legs (30,33)	0.25	1–1.25
All body parts (2–8,18,28,29,31,32)	0.25–3.25	0.5–4
Translation (1–3)	0.25–0.5	1–1.5
Jumps (4–8)	0.5–3.25	1.75–6.75
Energetic (1–8,18,23,26,28,29,31–33)	0.25–3.25	1–3.25
Non-energetic (9–17,19–22,24,25,27,30)	0.25–2	1.25–5.75
Rehab lower body (2–8,18,26–33)	0.25–3.25	1.25–3.25
Rehab upper body (9–11,13,16,19–25)	1–1.25	1.25–2.25
Security/Military (1–6,18,26,27)	0.25–2	1–5.75
Gaming jumps (4–8)	0.5–3.25	1.75–6.75
Gaming hits (20,23)	1	1.25–1.75
Gaming dance (12,18,19,22,28)	0.5–2	0.75–4
Sport/Wellness (1–33)	0.25–3.25	0.5–6.75

[Banos, O., Galvez, J. M., Damas, M., Pomares, H., & Rojas, I. (2014). Window size impact in human activity recognition. *Sensors*, 14(4), 6474–6499.]

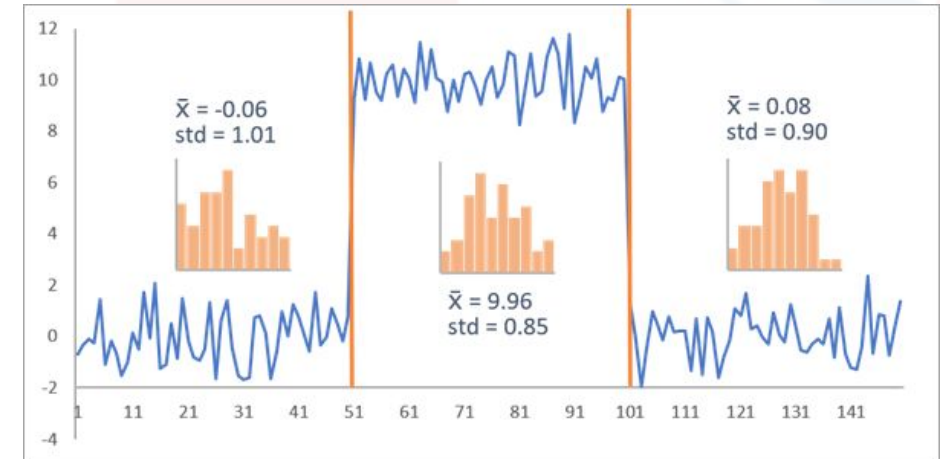
W_{min_size} -> to prioritize speed
 W_{max_perf} -> to prioritize accuracy

Dynamic Segmentation

- The drawback of sliding windows is that each segment may not completely represent an activity pattern
- Also, choosing the window size is not trivial
- There are techniques for ***dynamic segmentation***
 - The idea: using unsupervised AI methods to determine change points in sensor data streams
 - Intuitively, such changes indicate transitions between activities
- These methods are generally expensive from the computation point of view and are not adequate if there are real-time requirements

Dynamic Segmentation: how does it work

- Dynamic segmentation is often framed as an **optimization** problem that minimizes a cost function
- Each segment is indeed associated with a cost
 - e.g., avg, std, slope, etcetera
- The goal is to find the change points minimizing the costs of all the segments
- A penalty value is often added to the cost function in order to penalize the addition of each change point
 - high penalty may lead to too limited detected change points
 - low penalty may lead to creating too many change points



Segments Labeling

During training, annotations and sliding windows time intervals often do not match.

Most common approach: each window is labeled with the “prevalent” activity

Sensor data
windows



Ground truth

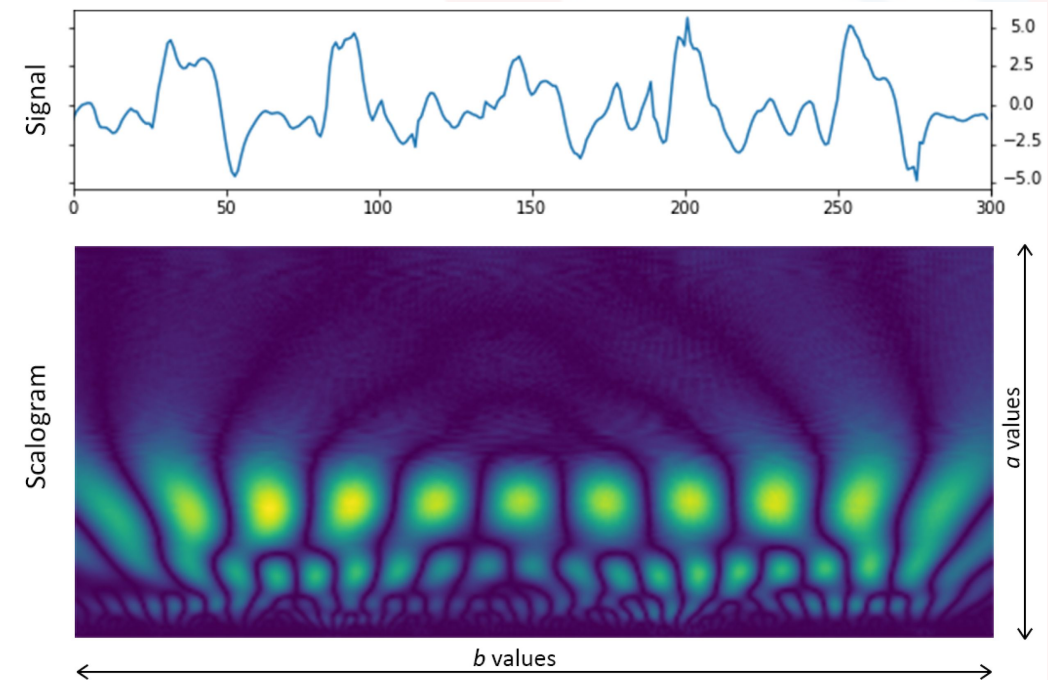


HAR with handcrafted feature extraction

Time-based vs Frequency-based

- ***Time-based features***
 - statistics directly computed on sensor data, they capture temporal information
- ***Frequency-based features***
 - statistics that capture properties of the frequency of sensor data

Note that features require to be computed on each axis of each sensor!



Examples of handcrafted features

Group	Methods
Time domain	Mean, standard deviation, variance, interquartile range (IQR), mean absolute deviation (MAD), correlation between axes, entropy, and kurtosis [19], [23]–[25], [27], [36], [45].
Frequency domain	Fourier Transform (FT) [27], [36] and Discrete Cosine Transform (DCT) [51].
Others	Principal Component Analysis (PCA) [35], [51], Linear Discriminant Analysis (LDA) [36], Autoregressive Model (AR), and HAAR filters [28].

[Lara, O. D., & Labrador, M. A. (2012). A survey on human activity recognition using wearable sensors. IEEE communications surveys & tutorials, 15(3), 1192-1209.]

HAR with automatic feature extraction

Automatic Feature Extraction

- The pre-processing step on raw data is still needed
 - cleaning and segmentation
- Pre-processed raw sensor data are directly provided to a deep neural network in charge of automatically extracting meaningful features during training
- During classification, the model will automatically extract features from unlabeled segments to perform classification!

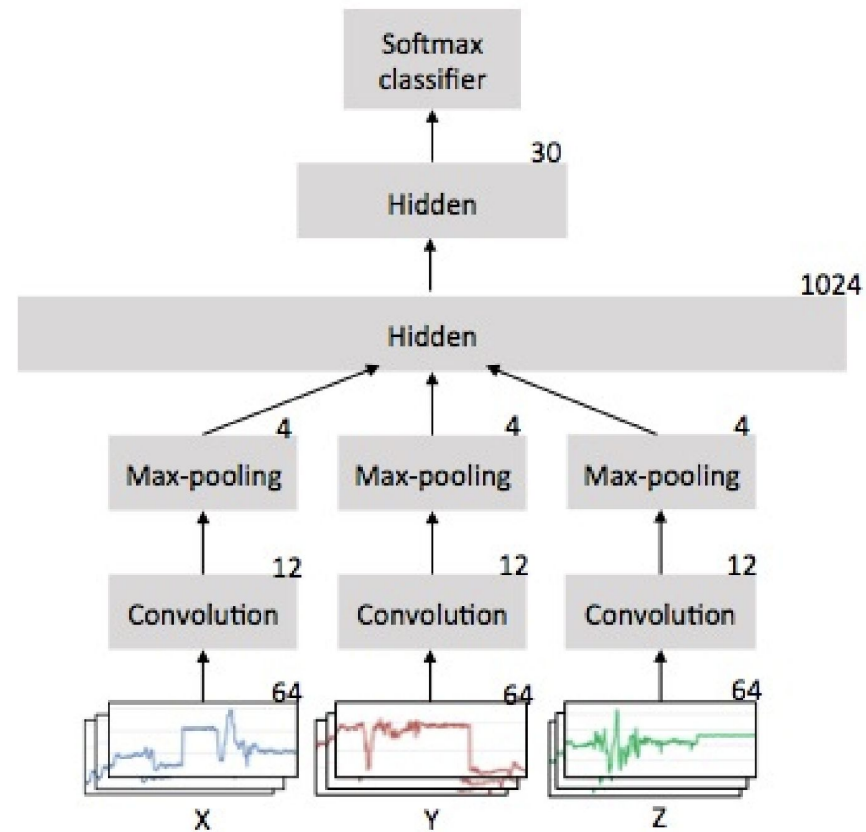
Convolutional Neural Network (CNN)

- Even though CNNs are designed for computer vision, they have advantages in processing time-series:
 - Local dependency: signals that are close are likely correlated
 - Scale-invariance: robustness to change in frequencies
- ***Drawback:*** CNNs do not fully capture temporal relationships

CNN: input adaptation

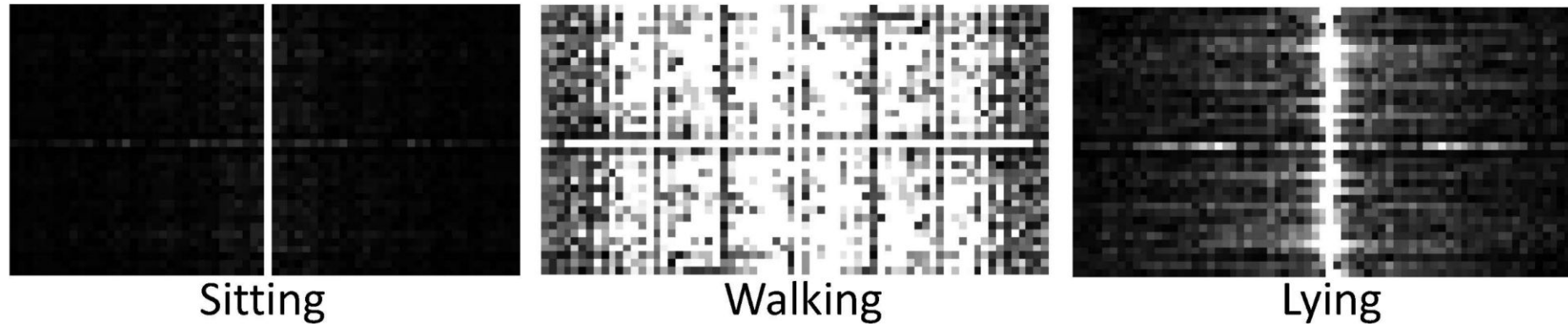
- A crucial step is adapting the input to the network
 - in HAR, the input is a multivariate time series, that is a set of 1D vectors
- Possible options:
 - single-head 1D CNN
 - multi-head 1D CNN, treating each dimension as a separate channel (e.g., each axis) and performing 1D convolution
 - transform the input into a 2D image and adopt 2D convolution

CNN with separate channels



Sensor data to images: an example

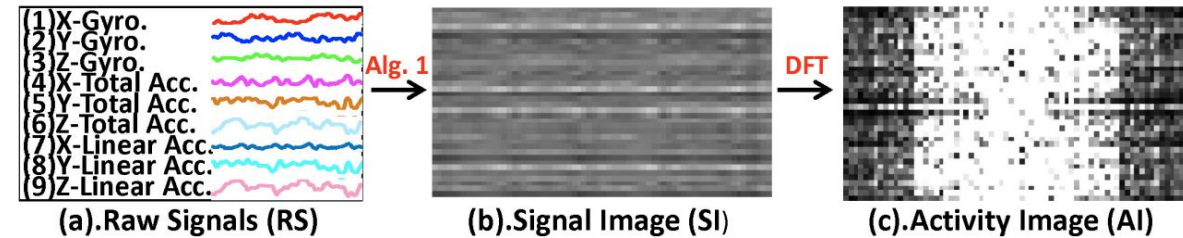
Given a window of multiple sensor streams, this method generates a 2D matrix and then compute 2D Discrete Fourier Transform.



Jiang, W., & Yin, Z. (2015, October). Human activity recognition using wearable sensors by deep convolutional neural networks. In *Proceedings of the 23rd ACM international conference on Multimedia* (pp. 1307-1310).

How images are generated in this approach?

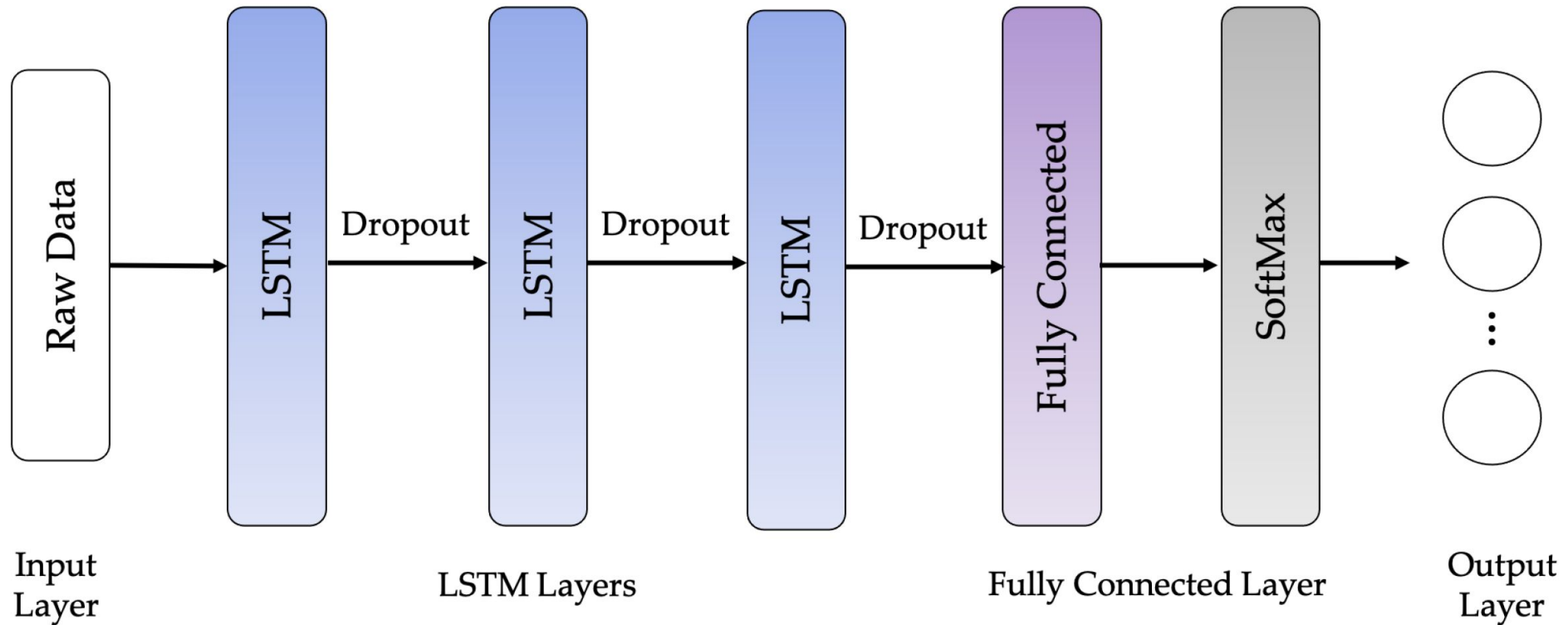
- Stacking row-by-row the pre-processed raw sensor data from the different time series to create a Signal Image (SI)
 - each time series is repeated so to make it adjacent to every other one
 - this allows CNN extracting hidden correlations between neighboring signals
- 2D Discrete Fourier Transform is applied to SI and its magnitude is chosen as Activity Image (AI)



Long-Short Term Memory (LSTM)

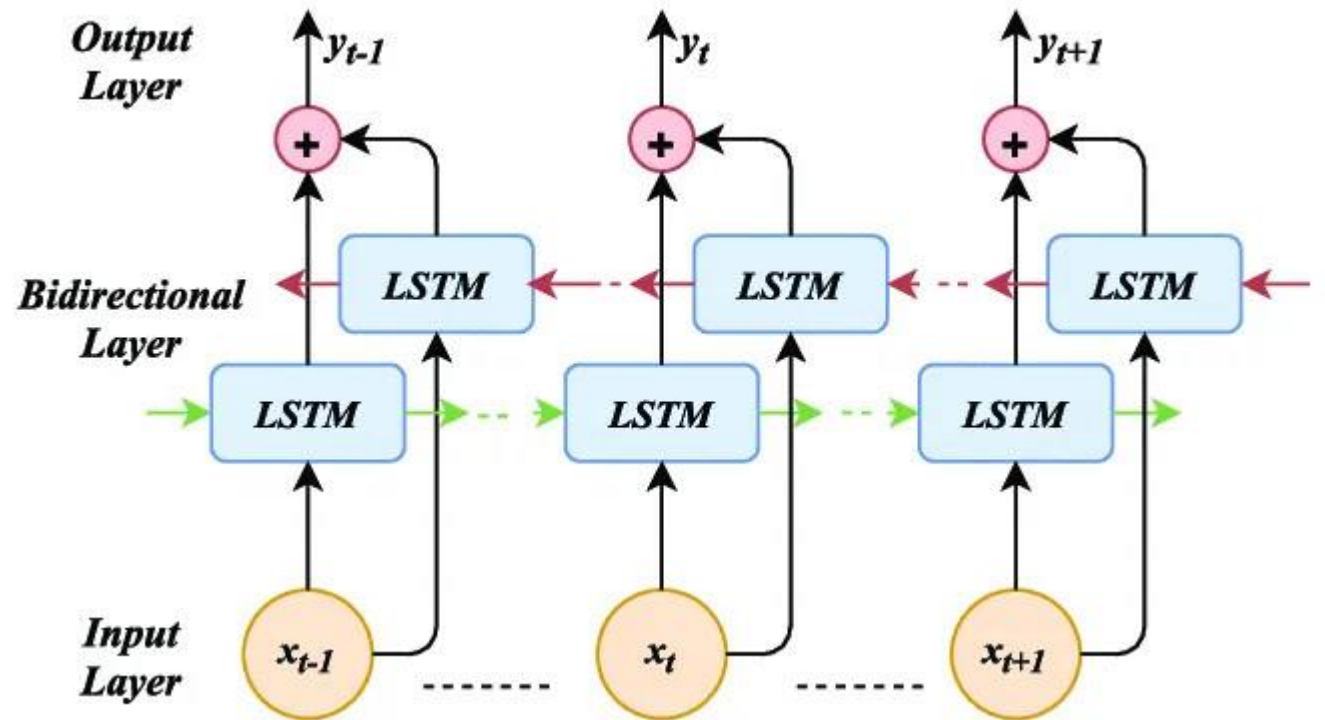
- Recurrent neural networks, and in particular LSTMs, can automatically extract strong temporal features from time series
- Hence they are widely used for HAR!
- ***Drawback:*** they may lead to overfitting on small datasets

A classic LSTM for HAR



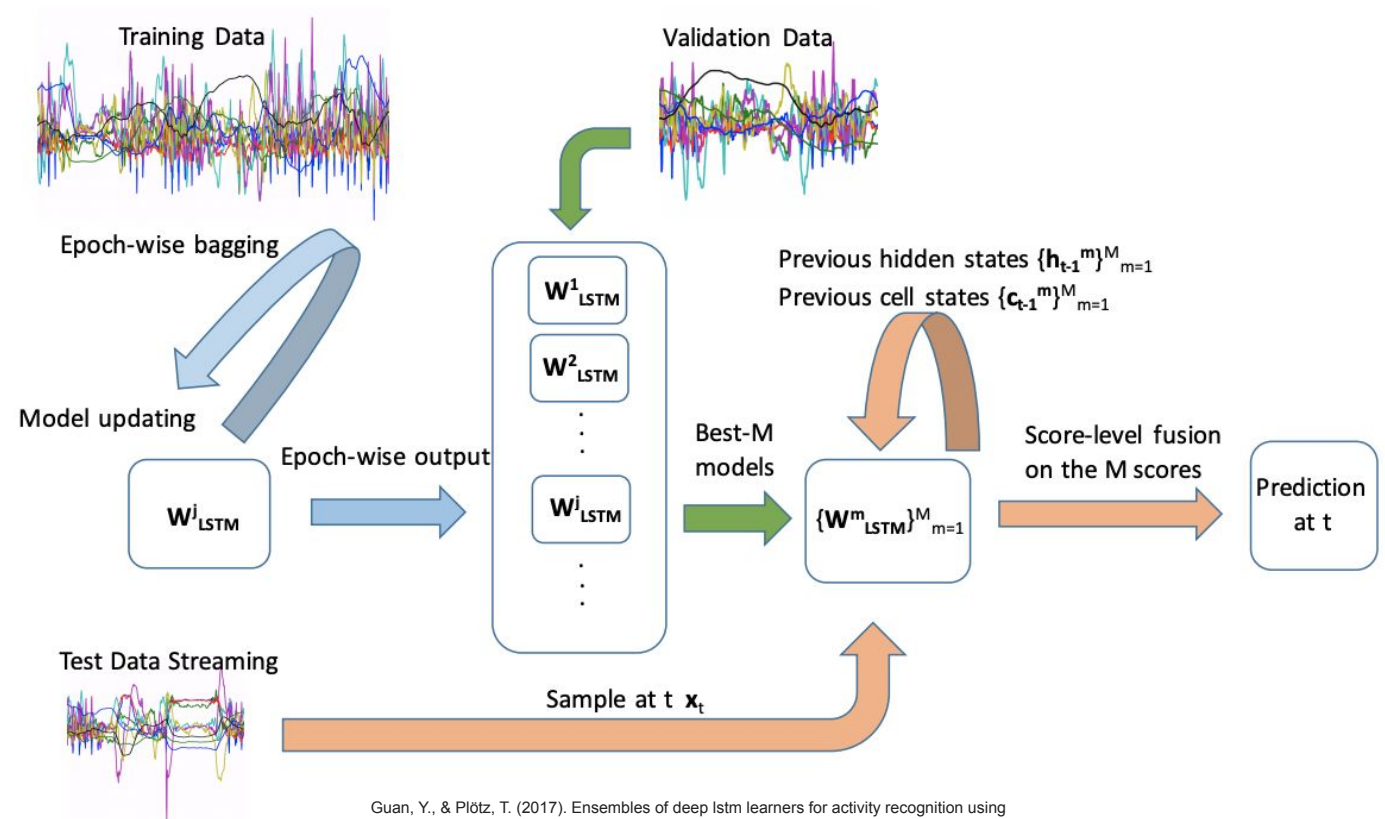
BiLSTM

- A special LSTM layer capable of capturing both **forward** and **backward** dependencies in the time series
- An additional LSTM unit is added to process the time series backwards
- Forward and backwards outputs are aggregated
- Better in capturing temporal properties of the time series, reducing overfitting problems!



LSTM ensembles

- During training, a pool of LSTM neural networks are trained at the same time
- Each LSTM is trained differently:
 - randomly selecting subsets of training data for parameter estimation.
 - employing different loss functions for model training, which leads to variations in resulting LSTM models
- Goal: increase robustness to noise and unbalanced data

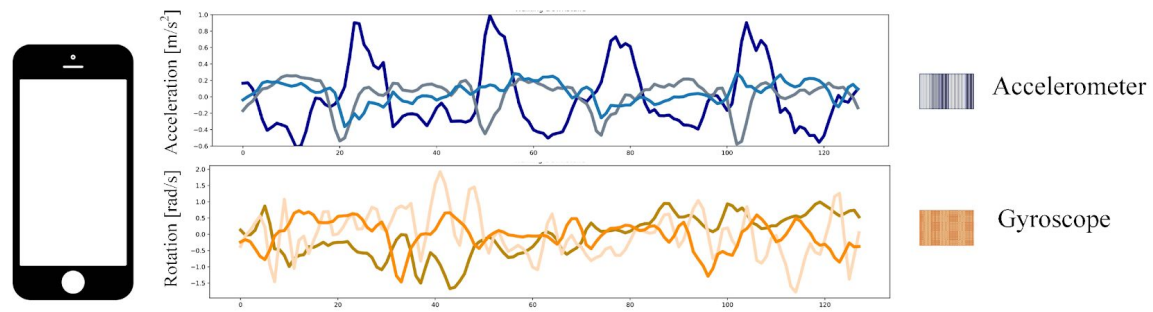
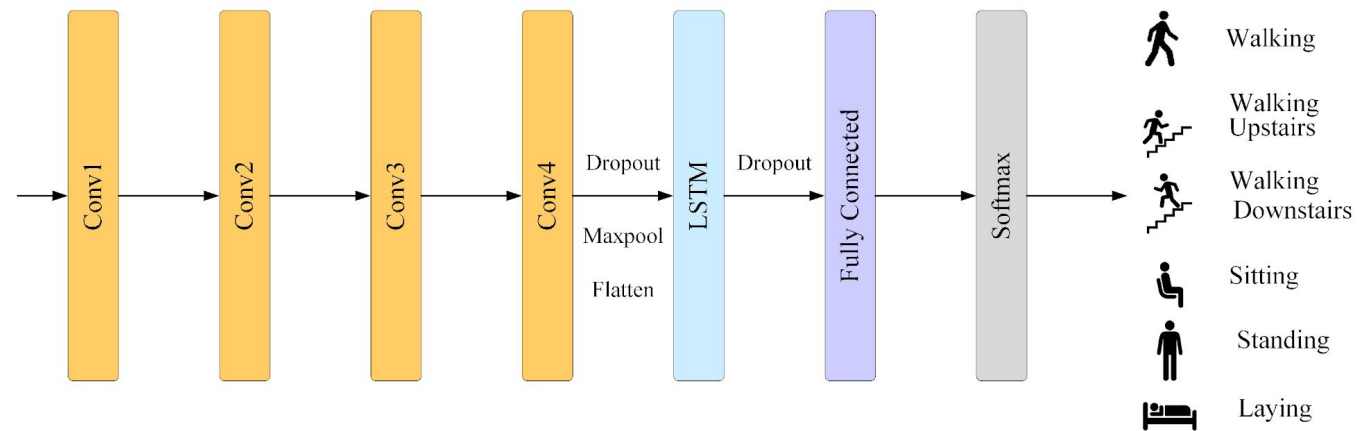


Guan, Y., & Plötz, T. (2017). Ensembles of deep lstm learners for activity recognition using wearables. *Proceedings of the ACM on interactive, mobile, wearable and ubiquitous technologies*, 1(2), 1-28.

Hybrid Approaches

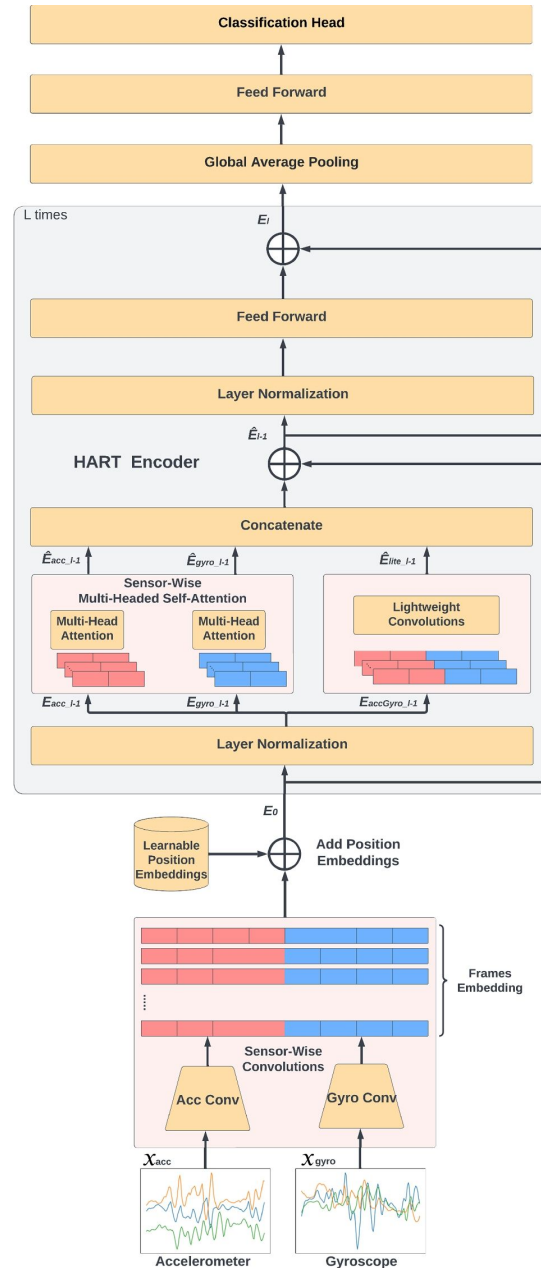
- Among the most effective solutions for supervised HAR, the combination of convolutional and recurrent layers is particularly effective
- Convolutional Layers are first used to extract meaningful features from raw sensor data
- Recurrent Layers are then used to capture temporal relationships!

A Hybrid CNN-LSTM network



A Transformer for HAR

Here, we see an **encoder-only** transformer (typically adopted for classification tasks, even in NLP)



Ek, S., Portet, F., & Lalanda, P. (2023). Transformer-based models to deal with heterogeneous environments in Human Activity Recognition. *Personal and Ubiquitous Computing*, 27(6), 2267-2280.

The Deployment on Mobile Devices

Offline VS Online Recognition

- *Offline* recognition:
 - devices only collect data, that are stored to be processed in another phase
 - this may be useful if real-time is not a requirement, in favor of accuracy
 - e.g., behavior analysis
- *Online* recognition:
 - recognizing activities in real-time
 - e.g., to support a context-aware applications!

Online recognition: where it should run?

- Server-side:
 - **pro:** high computational capabilities
 - **cons:** overhead in network communication, reduced privacy
- Client-side:
 - **pro:** good support to real-time recognition, better privacy
 - **cons:** complex models can not run on efficiently on resource-constrained devices

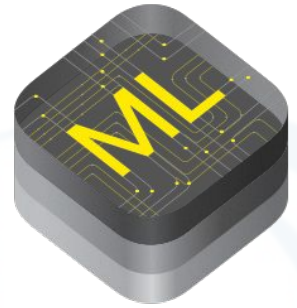
How to deploy HAR models on mobile devices?

- The model should be created *offline* using a training set
- Libraries for mobile programming languages are in charge of loading the model and use it for HAR
- The same SDK should be used to *create* and *deploy* the model

SDKs for ML on mobile devices

The most commonly used are:

- Weka (Android)
- CoreML (iOS)
- TensorFlow Lite (both Android and iOS)



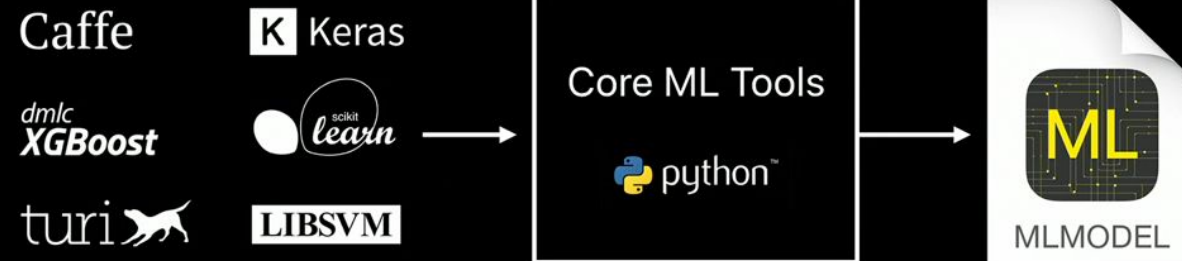
TensorFlow

Deploying the model

- The HAR recognition model can be trained *offline*
 - for instance, using Python libraries!
- When the model is ready, it can be converted to a format suitable for the specific Mobile SDK being used
- Then it can be deployed on the application

An example with CoreML

Convert to Core ML



After the deployment

After the deployment of the model on the mobile device:

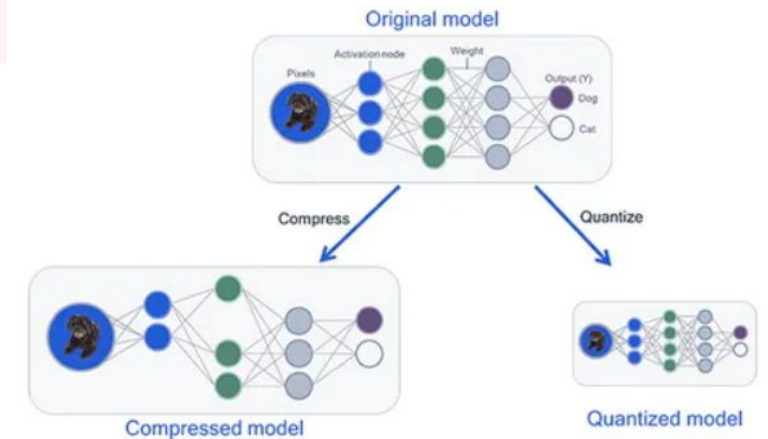
- The mobile device collects sensors data stream
- It performs segmentation and pre-processing
 - with the same parameters used during training!
- Each segment is classified by the model to obtain the activity label

Efficiency

- Mobile/wearable devices usually have computational constraints
- Deep learning models may also significantly impact energy usage, with negative impact on usability
- Hence, the models should be “simplified” in order to run on such devices!

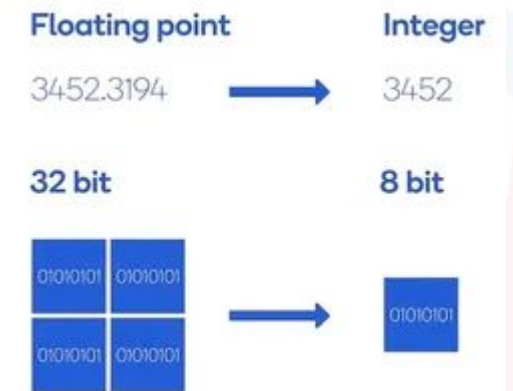
How to create more efficient models?

- Design simpler models from the begin, or...
- Reducing the size of existing complex models
 - compressing the model into a smaller one
 - pruning unimportant parts of the model (e.g., neurons/weights with minor impact on accuracy)
 - merge components of the model (e.g., sharing weights for similar edges)
 - using knowledge distillation
 - reducing the resolution of weights storage (quantization)
- Major problem: guaranteeing a trade-off between recognition rate and efficiency!
- Remember: compression and quantization are two different processes
 - ...but they can be combined!



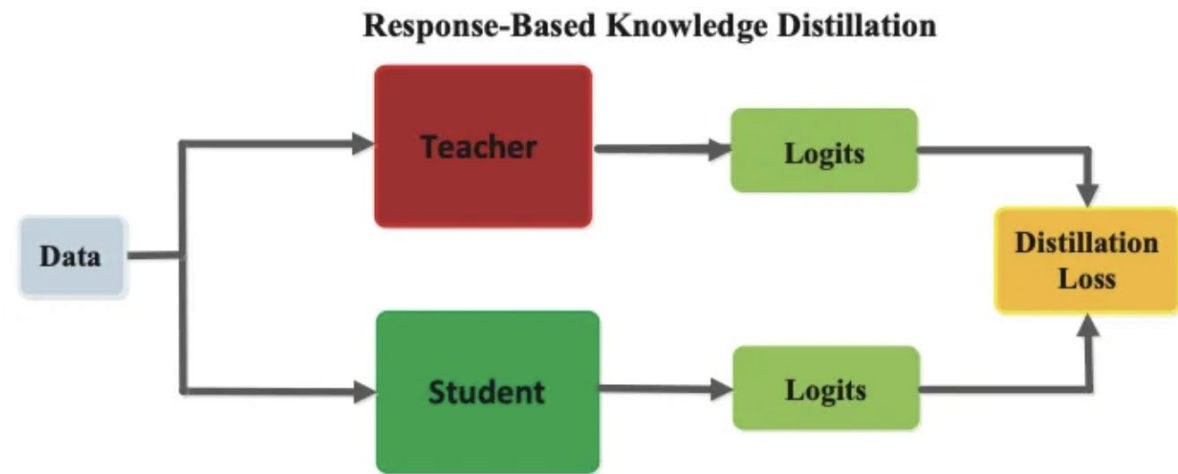
Quantization

- Standard approaches:
 - reducing precision of floating points (e.g., from 32 bits to 16 bits representation)
 - switching to integers or fixed points numbers (e.g., 8 bit)
 - goal: reducing memory and computational requirements!
- Sometimes this is not enough for low-power micro-controllers
- Other advanced approaches:
 - **sub-byte quantization**: using less than 8 bits, challenging since data are stored at byte level in the HW
 - **mixed precision**: allowing different layers to use different bit-widths, avoiding the potential accuracy drops caused by a fixed low-bit-width solution. Better trade-off between accuracy and model size, but way more difficult to design



A compression approach: Knowledge Distillation

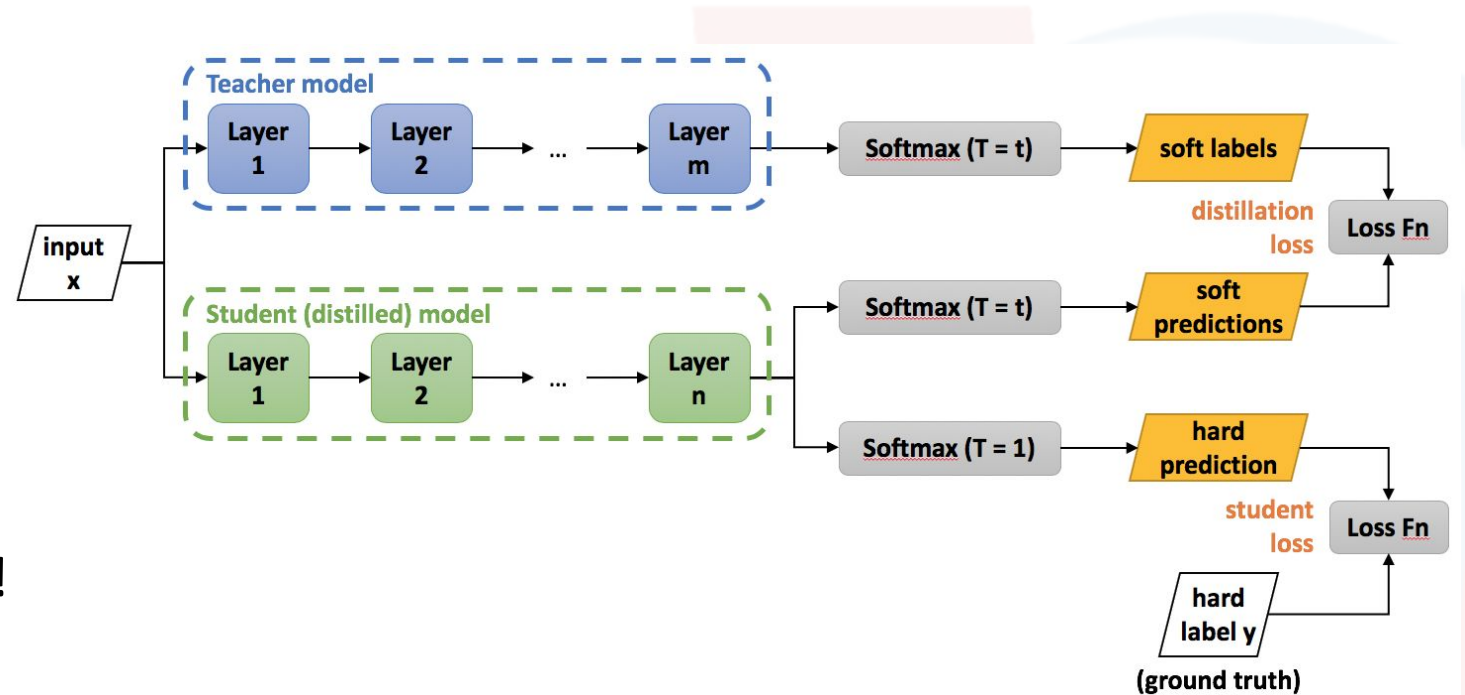
- Idea: training a “small” model (student) relying on the knowledge of a “bigger” model (teacher)
- The ***distillation loss*** considers the probability distribution over the possible activities of the teacher to transfer knowledge on the student



<https://towardsdatascience.com/on-the-edge-deploying-deep-applications-on-constrained-devices-f2dac997dd4d>

Knowledge Distillation: how does it work

- The teacher (the larger model) is used to obtain its softmax output (**soft labels**)
- The distillation loss forces the student (the compressed smaller model) in generating similar probability distributions on the same data
- The student loss is the target HAR task (e.g., cross-entropy loss)
- The two losses are jointly learned!



Thank you